DRAFT FOR APPROVAL

# StockWise
# Multi-store Firestore Schema

A review-ready, tenant-isolated database blueprint for the Android and iOS application. It covers stores, staff membership, users, expiring inventory, promotions, carts, orders, waste analytics, notifications, access control, and Firebase Storage.

**Recommended foundation**

Cloud Firestore with each store as a tenant boundary, plus a reusable product catalog and separate inventory batches inside that store. Users may work in several stores with a different role in each one.

| Platform | Tenancy | Status |
| --- | --- | --- |
| Android + iOS only | Multiple isolated stores | Awaiting approval |

Prepared 09 September 2026

# 1. What the app needs

The schema is based on a screen-by-screen scan of the current Flutter project.

| Area | Observed data requirement |
| --- | --- |
| **Authentication and profile** | Firebase email/password sign-in, store selection, profile data, notification preferences, and logout. |
| **Store administration** | Multiple isolated stores, store identity/address/settings, and per-store owner, manager, or staff membership. |
| **Inventory administration** | Create/edit products, photo, category, barcode, quantity, expiry date, promotion price, and expiry alerts. |
| **Stock operations** | Expiring-soon dashboard, inventory list, mark consumed, adjust stock, delete/close a batch. |
| **Customer shopping** | Promotion catalog, search, batch expiry display, cart quantities, and checkout. |
| **Orders** | To Pay, To Ship, To Receive, Completed; pickup/delivery; receiver snapshot; totals and payment state. |
| **Waste reporting** | Waste by category and month, requiring a durable stock-event history rather than hard-coded chart totals. |
| **Notifications** | Expiry warnings, consumed-stock activity, order updates, and optional Android/iOS push delivery. |

### Important security finding

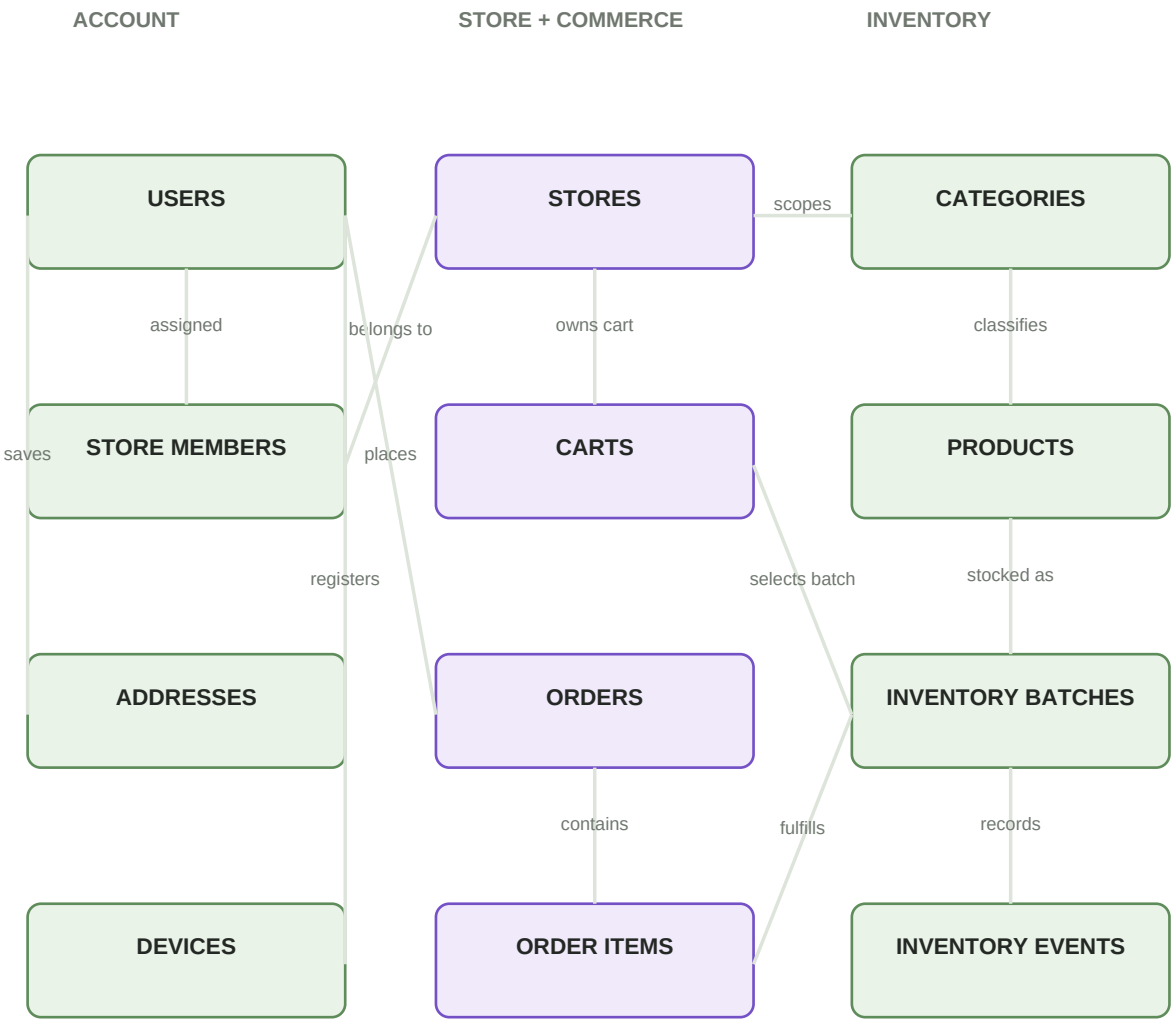
The current prototype identifies the admin by the exact email admin@stockwise.com. Production authorization should instead check stores/{storeId}/members/{uid}.role. A platform-wide custom claim is reserved only for a trusted super administrator.

### Category mismatch to resolve

The item editor currently uses Dairy, Bakery, Produce, and Household, while waste reporting uses Dairy, Vegetables, Fruits, Meat, and Others. A categories collection creates one canonical list for both screens.

# 2. Relationship map

A store is the tenant boundary. Its catalog, inventory, carts, orders, reports, and alerts remain isolated from every other store.




### Why carts and orders point to batches

A customer selects one store before shopping. Their cart, order, expiry date, promotional price, and stock reservation all remain inside that store.

# 3. Store tenancy and membership

Store roles are not global. The same person can manage Store A, work as staff in Store B, and shop as a customer elsewhere.

**stores/{storeId}** — Tenant root and store identity

```
name: string
code: string
logoPath: string?
ownerUid: uid
phone: string?
email: string?
address: {line1, line2?, city, state, postalCode, countryCode}
timezone: string
currency: 'MYR'
status: 'active' | 'suspended' | 'closed'
createdAt: timestamp
updatedAt: timestamp
```

**stores/{storeId}/members/{uid}** — Per-store privileged access

```
storeId: reference/id
userId: uid
role: 'owner' | 'manager' | 'staff'
permissions: string[]?
isActive: bool
invitedBy: uid?
joinedAt: timestamp
updatedAt: timestamp
```

**users/{uid}** — Global account profile and preferences

```
email: string
displayName: string
phone: string?
photoPath: string?
activeStoreId: reference/id?
preferences.expiryAlerts: bool
preferences.orderUpdates: bool
preferences.promotions: bool
createdAt: timestamp
updatedAt: timestamp
```

> **Tenant rule**
>
> Every store-owned document stays below stores/{storeId}. Store staff access is authorized against that store's membership document. Customers do not need staff membership to shop or place their own orders.

# 4. Accounts and per-store catalog

### users/{uid}/addresses/{addressId}

Saved checkout destinations

```
label: string
recipientName: string
phone: string
addressLine1: string
addressLine2: string?
city: string
state: string
postalCode: string
countryCode: string
isDefault: bool
```

### stores/{storeId}/categories/{categoryId}

Canonical categories for one store

```
storeId: reference/id
name: string
slug: string
sortOrder: number
isActive: bool
createdAt: timestamp
updatedAt: timestamp
```

### stores/{storeId}/products/{productId}

Reusable product master inside one store

```
storeId: reference/id
name: string
nameLowercase: string
description: string?
categoryId: reference/id
barcode: string?
imagePath: string?
unitLabel: string
basePriceCents: integer
currency: 'MYR'
isActive: bool
createdBy: uid
createdAt: timestamp
updatedAt: timestamp
```

> **Money rule**
>
> Store monetary values as integer cents. Example: RM 20.50 is stored as 2050. This avoids floating-point rounding errors.

# 5. Per-store inventory and waste

**stores/{storeId}/inventoryBatches/{batchId}**

A separately expiring delivery for one store

```
storeId: reference/id
productId: reference/id
productName: string (query/display copy)
productNameLowercase: string
categoryId: reference/id (query copy)
imagePath: string? (display copy)
unitLabel: string (display copy)
expiryAt: timestamp
quantityOnHand: integer
quantityReserved: integer
status: 'active' | 'depleted' | 'expired' | 'discarded'
promotionEnabled: bool
promotionPriceCents: integer?
promotionStartsAt: timestamp?
promotionEndsAt: timestamp?
notifyOnExpiry: bool
createdBy: uid
createdAt: timestamp
updatedAt: timestamp
```

**stores/{storeId}/inventoryEvents/{eventId}**

Append-only store audit trail and waste source

```
storeId: reference/id
batchId: reference/id
productId: reference/id
categoryId: reference/id
eventType: 'received' | 'adjusted' | 'reserved' | 'released' | 'sold' | 'consumed' | 'discarded' |
'expired'
quantity: integer
quantityDelta: integer
reason: string?
actorUid: uid?
orderId: reference/id?
occurredAt: timestamp
```

### Waste calculation

Only discarded and expired events feed the waste dashboard. Sales and normal consumption reduce stock but are not counted as waste. Monthly totals and category composition are derived from this event history.

### Atomic stock rule

Cart reservation, order placement, release, and sale must update quantities in a Firestore transaction or trusted backend function so stock cannot become negative.

# 6. Per-store cart and orders

**stores/{storeId}/carts/{uid}** — One customer cart inside one store

```
storeId: reference/id
ownerUid: uid
updatedAt: timestamp
```

**stores/{storeId}/carts/{uid}/items/{batchId}** — Exact store batch selected by the customer

```
storeId: reference/id
batchId: reference/id
productId: reference/id
productName: string
imagePath: string?
expiryAt: timestamp
quantity: integer
unitPriceCents: integer
addedAt: timestamp
updatedAt: timestamp
```

**stores/{storeId}/orders/{orderId}** — Store order header, receiver, payment, and totals

```
storeId: reference/id
orderNumber: string (unique within store)
userId: uid
status: 'to_pay' | 'to_ship' | 'to_receive' | 'completed' | 'cancelled'
paymentStatus: 'unpaid' | 'paid' | 'refunded'
fulfillmentType: 'pickup' | 'delivery'
receiver: {name, phone, email, addressLine1, addressLine2?, city, state, postalCode, countryCode}
payment: {methodLabel, provider?, providerReference?}
subtotalCents: integer
shippingFeeCents: integer
totalCents: integer
currency: 'MYR'
placedAt: timestamp
updatedAt: timestamp
completedAt: timestamp?
```

**stores/{storeId}/orders/{orderId}/items/{itemId}** — Immutable store receipt snapshot

```
storeId: reference/id
productId: reference/id
batchId: reference/id
productName: string
imagePath: string?
expiryAt: timestamp
unitLabel: string
quantity: integer
unitPriceCents: integer
lineTotalCents: integer
```

### Payment boundary

Store only payment state and provider-generated references. Never store bank credentials, passwords, card data, or e-wallet secrets in Firestore.

# 7. Per-store notifications, settings, and files

**stores/{storeId}/notifications/{notificationId}** — Store alert record for one recipient

```
storeId: reference/id
recipientUid: uid
type: 'expiry_warning' | 'stock_consumed' | 'order_status' | 'promotion'
title: string
body: string
relatedCollection: string?
relatedDocumentId: string?
readAt: timestamp?
createdAt: timestamp
scheduledFor: timestamp?
```

**users/{uid}/devices/{deviceId}** — Android/iOS push notification registration

```
fcmToken: string
platform: 'android' | 'ios'
enabled: bool
lastSeenAt: timestamp
```

**stores/{storeId}/settings/inventory** — Store-specific expiry behavior

```
expiryWarningDays: [2, 3, 7]
```

**stores/{storeId}/settings/commerce** — Store-specific checkout configuration

```
currency: 'MYR'
deliveryFeeCents: integer
minimumOrderCents: integer?
```

| Firebase Storage path | Purpose |
|---|---|
| `stores/{storeId}/product-images/{productId}/...` | Store catalog and promotion artwork |
| `user-avatars/{uid}/...` | Profile photos |
| `stores/{storeId}/order-attachments/{orderId}/...` | Optional store order documents |

# 8. Multi-store access control and indexes

- **Users:** Read and update their own global profile, addresses, preferences, and devices.

- **Stores:** Active stores are discoverable; sensitive settings require active store membership.

- **Membership:** Owners manage managers/staff. Users may read their own membership. Store roles are never client-self-assigned.

- **Catalog:** Customers read active store products. Store owner, manager, or authorized staff create and update.

- **Inventory:** Authorized store members write. Customers read only sale-safe batches in the selected store.

- **Carts:** Owner-only read/write, constrained to the selected store.

- **Orders:** Customers create/read their own store orders. Authorized store members read that store's orders.

- **Notifications:** Recipients and authorized store staff read appropriate alerts. Trusted backend creates system alerts.

- **Devices:** Owner-only registration. Tokens are never publicly readable.

> **Role enforcement**
>
> Authorize normal administration with stores/{storeId}/members/{uid}.role. Reserve a Firebase Authentication custom claim only for a rare platform-wide super administrator. This avoids granting one user admin rights over every store.

## Required composite indexes

| Collection | Fields |
|---|---|
| `members (group)` | `userId ASC, isActive ASC` |
| `inventoryBatches` | `status ASC, expiryAt ASC` |
| `inventoryBatches` | `promotionEnabled ASC, status ASC, expiryAt ASC` |
| `orders` | `userId ASC, status ASC, placedAt DESC` |
| `orders` | `status ASC, placedAt DESC` |
| `notifications` | `recipientUid ASC, createdAt DESC` |
| `inventoryEvents` | `eventType ASC, occurredAt DESC` |
| `inventoryEvents` | `categoryId ASC, eventType ASC, occurredAt DESC` |

# 9. Approval checklist

Approve the multi-store foundation before any Firestore collections, rules, indexes, Storage paths, or app repositories are wired.

| | |
|---|---|
| [ ] | **Multi-store tenant isolation**<br>Each store owns its catalog, batches, carts, orders, reports, alerts, and settings. |
| [ ] | **Per-store staff roles**<br>A user can be owner, manager, or staff in different stores without receiving global admin access. |
| [ ] | **Product and batch separation**<br>Products are reusable masters; batches own expiry, available stock, and promotional price. |
| [ ] | **One cart per user per store**<br>Items from different stores are never mixed in one checkout. |
| [ ] | **Canonical categories**<br>Dairy, Bakery, Vegetables, Fruits, Meat, Household, and Others. |
| [ ] | **Membership-based security**<br>The admin email shortcut is replaced with store membership checks. |
| [ ] | **Event-derived waste analytics**<br>Only expired and discarded events count as waste. |
| [ ] | **Payment metadata only**<br>No sensitive financial credentials are stored. |

> **Approval phrase**
>
> Reply with: Approve Multi-store Schema v1. If you want changes, list them before approval and the schema can be revised without touching Firebase.

## Implementation after approval

- Add Cloud Firestore, Firebase Storage, and optional Firebase Messaging dependencies.
- Create store selection, membership-aware navigation, typed models, repositories, and transactional stock operations.
- Deploy security rules and composite indexes.
- Replace hard-coded UI data with live streams and role-safe actions.
- Verify Android and iOS flows only.